



Introduction Laravel

Contents

- What is Laravel?
- Advantages of Laravel
- Features of Laravel
- History of Laravel
- Install Laravel on Windows
- Directory Structure
- Best Practices

What is Laravel?

- Laravel is an open-source PHP framework.
- Laravel is a PHP framework that uses the MVC architecture.



History of Laravel

Version ↕	Release date ↕	PHP version ↕
1.0	June 2011	
2.0	September 2011	
3.0	February 22, 2012	
3.1	March 27, 2012	
3.2	May 22, 2012	
4.0	May 28, 2013	≥ 5.3.0
4.1	December 12, 2013	≥ 5.3.0
4.2	June 1, 2014	≥ 5.4.0

Laravel 1

It featured a custom ORM (Eloquent); closure-based routing; a module system for extension; and helpers for forms, validation, authentication, and more.

Laravel 2, 3

They introduced controllers, unit testing, a command-line tool, an inversion of control (IoC) container, Eloquent relationships, and migrations., and more.

Laravel 4

Laravel 4 also introduced queues, a mail component, facades, and database seeding. And because Laravel was now relying on Symfony components.

History of Laravel

<https://en.wikipedia.org/wiki/Laravel>

5.0	February 4, 2015	August 4, 2015	February 4, 2016	≥ 5.4.0
5.1 LTS	June 9, 2015	June 9, 2017	June 9, 2018	≥ 5.5.9
5.2	December 21, 2015	June 21, 2016	December 21, 2016	≥ 5.5.9
5.3	August 23, 2016	February 23, 2017	August 23, 2017	≥ 5.6.4
5.4	January 24, 2017	July 24, 2017	January 24, 2018	≥ 5.6.4
5.5 LTS	August 30, 2017	August 30, 2019	August 30, 2020	≥ 7.0.0
5.6	February 7, 2018	August 7, 2018	February 7, 2019	≥ 7.1.3
5.7	September 4, 2018	March 4, 2019	September 4, 2019	≥ 7.1.3
5.8	February 26, 2019	August 26, 2019	February 26, 2020	≥ 7.1.3
6 LTS	September 3, 2019	January 25, 2022	September 6, 2022	7.2 – 8.0 ^[25]
7	March 3, 2020 ^[26]	October 6, 2020	March 3, 2021	7.2 – 8.0 ^[20]
8	September 8, 2020	July 26, 2022	January 24, 2023	7.3 – 8.1 ^[27]
9	February 8, 2022 ^[25]	August 8, 2023	February 6, 2024	8.0 – 8.2 ^[25]
10	February 14, 2023	August 6, 2024	February 4, 2025	8.1 – 8.3 ^[21]
11	March 12, 2024	September 3, 2025	March 12, 2026	8.2 – 8.4 ^[28]
12	February 24, 2025	August 13, 2026	February 24, 2027	≥ 8.2 ^[28]
13	February-March 2026			

Legend: Unsupported Supported Latest version Future version



MVC

- MVC is divided into three letters shown below:
 - M: 'M' stands for Model.

A model is a class that deals with a database.
 - V: 'V' stands for View.

A view is a class that deals with an HTML. Everything that we can see on the application in the browser is the view or the representation.
 - C: 'C' stands for Controller.

A controller is the middle-man that deals with both model and view. A controller is the class that retrieves the data from the model and sends the data to the view class.



Advantages of Laravel

Creating authorization and authentication System

Integration with tools

Mail Service integration

Handling exception and configuration error

Automation testing work

Separation of business logic code from presentation code

Fixing most common Technical Vulnerabilities

Scheduling tasks configuration and management



Advantages of Laravel

Creating authorization and authentication System

Every owner of the web application makes sure that unauthorized users do not access secured or paid resources. It provides a simple way of implementing authentication. It also provides a simple way of organizing the authorization logic and control access to resources.

Integration with tools

Laravel is integrated with many tools that build a faster app. It is not only necessary to build the app but also to create a faster app. Integration with the caching back end is one of the major steps to improve the performance of a web app. Laravel is integrated with some popular cache back ends such as **Redis**, and **Memcached**.

Mail Service integration

Laravel is integrated with the Mail Service. This service is used to send notifications to the user's emails. It provides a clean and simple API that allows you to send the email quickly through a local or cloud-based service of your choice.



Advantages of Laravel

Handling exception and configuration error

The manners in which the software app handles the errors have a huge impact on the user's satisfaction and the app's usability. The organization does not want to lose their customers, so for them, Laravel is the best choice. In Laravel, error and exception handling is configured in the new Laravel project.

Automation testing work

We know that automation testing is less time-consuming than manual testing, so automation testing is preferred over the manual testing. Laravel is developed with testing in mind.

Separation of business logic code from presentation code

A bug can be resolved by the developers faster if the separation is provided between the business logic code and presentation code. We know that Laravel follows the **MVC architecture**, so separation is already done.



Advantages of Laravel

Fixing most common Technical Vulnerabilities

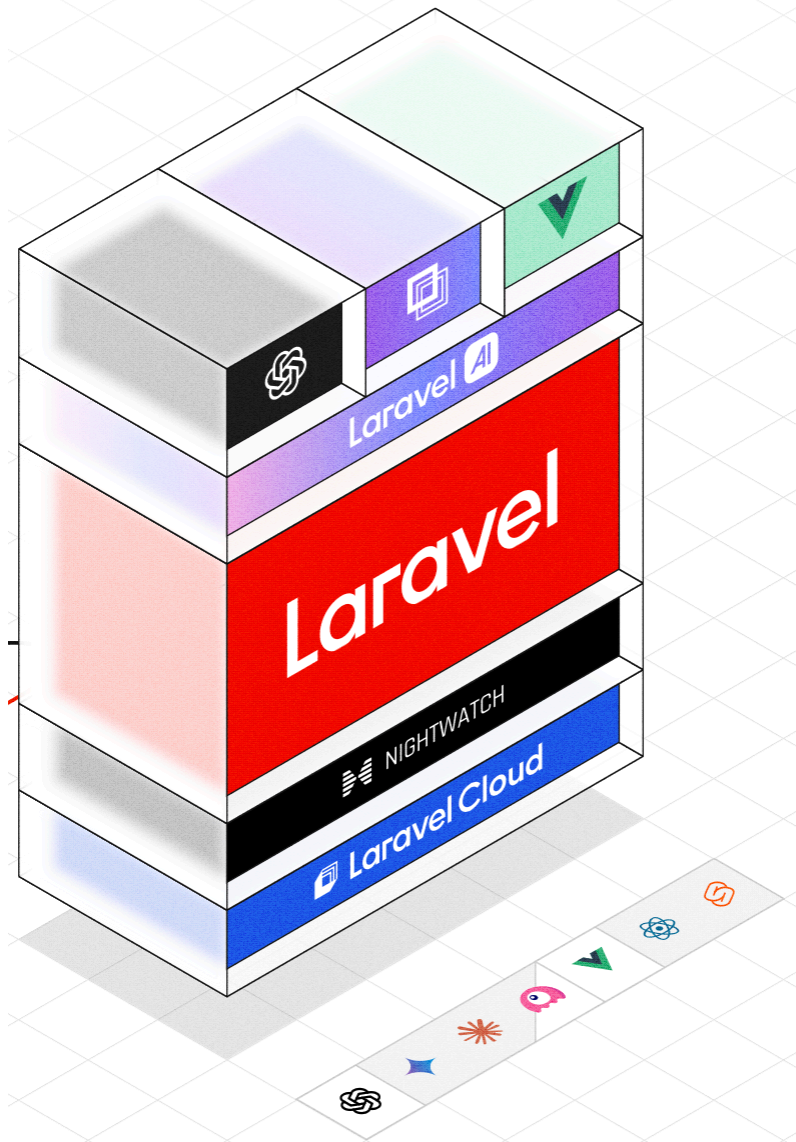
The **security vulnerability** is the most important example in web application development. An American organization, i.e., OWASP Foundation, defines the most important security vulnerabilities such as SQL injection, cross-site request forgery, cross-site scripting, etc. Developers need to consider these vulnerabilities and fix them before delivery. Laravel is a secure framework as it protects the web application against all the security vulnerabilities.

Scheduling tasks configuration and management

Laravel command scheduler defines a command schedule which requires a single entry on the server.



Laravel 12+





Install Laravel on Windows

- Install Composer
- Installing Laravel
 - First, download the Laravel installer using Composer:

```
composer global require laravel/installer
```

- Once installed, the laravel new command will create a fresh Laravel installation in the directory you specify.

```
laravel new example-app
```



Install Laravel on Windows

- Run example-app

```
cd example-app  
npm install && npm run build  
composer run dev
```



Configuration

■ Environment Configuration

- Your .env file should not be committed to your application's source control, since each developer / server using your application could require a different environment configuration.
- Furthermore, this would be a security risk in the event an intruder gains access to your source control repository, since any sensitive credentials would get exposed.



Hiding Environment Variables

```
return [  
    // ...  
  
    'debug_hide' => [  
        '_ENV' => [  
            'APP_KEY',  
            'DB_PASSWORD',  
        ],  
        '_SERVER' => [  
            'APP_KEY',  
            'DB_PASSWORD',  
        ],  
        '_POST' => [  
            'password',  
        ],  
    ],  
  
    // ...  
];
```

- You may do this by updating the debug_hide option in your **config/app.php** configuration file.



Accessing Configuration Values

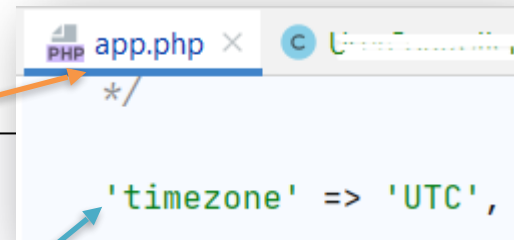
- Using the global **config** helper
- The configuration values may be accessed using "dot" syntax, which includes the name of the file and option you wish to access

```
//get timezone  
$value = config('app.timezone');
```

```
// Retrieve a default value if the configuration value does not exist...  
$value = config('app.timezone', 'Asia/Seoul');
```

```
//To set configuration values at runtime, pass an array to the config  
helper:
```

```
config(['app.timezone' => 'America/Chicago']);
```



Directory Structure

- ▼ Laravel
 - ▼ app
 - ▶ Console
 - ▶ Exceptions
 - ▶ Http
 - ▶ Models
 - ▶ Providers
 - ▶ bootstrap
 - ▶ config
 - ▶ database
 - ▶ public
 - ▼ resources
 - ▶ css
 - ▶ js
 - ▶ lang
 - ▶ views
 - ▶ routes
 - ▶ storage
 - ▶ tests
 - ▶ vendor
 - ▶ .editorconfig
 - ▶ .env
 - ▶ .env.example

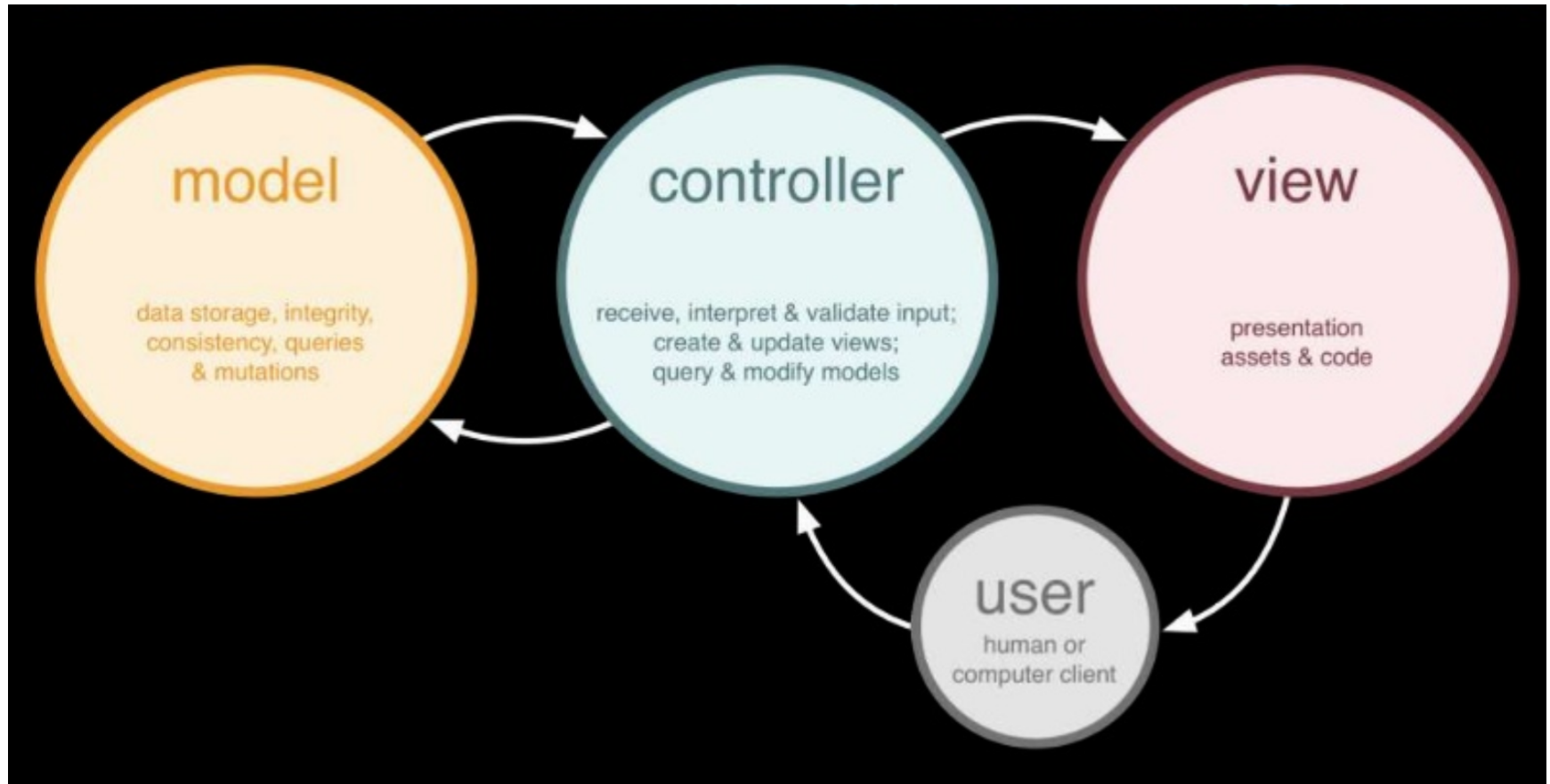
- ▼ app
 - ▶ Console
 - ▶ Exceptions
 - ▶ Http
 - ▼ Models
 - User.php
 - ▶ Providers

- ▼ database
 - ▶ factories
 - ▶ migrations
 - ▶ seeders
 - ▶ .gitignore

- ▼ storage
 - ▼ app
 - ▶ public
 - ▶ .gitignore
 - ▶ framework
 - ▶ logs



MVC

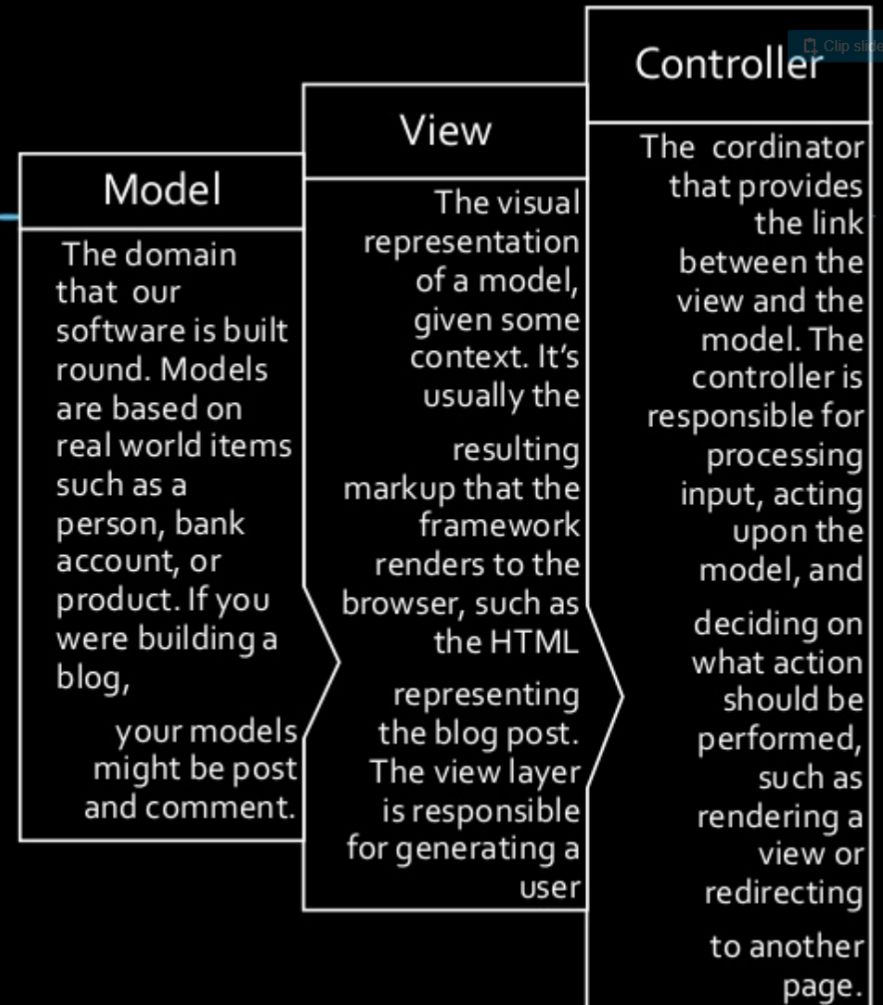
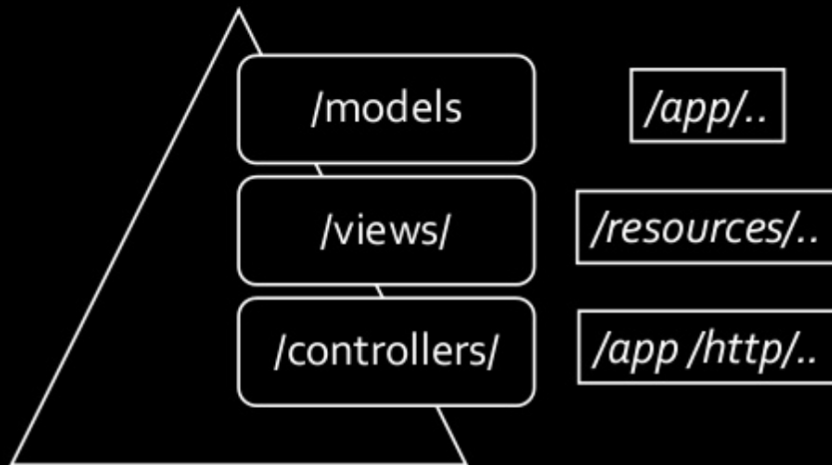




MVC

MVC explanation

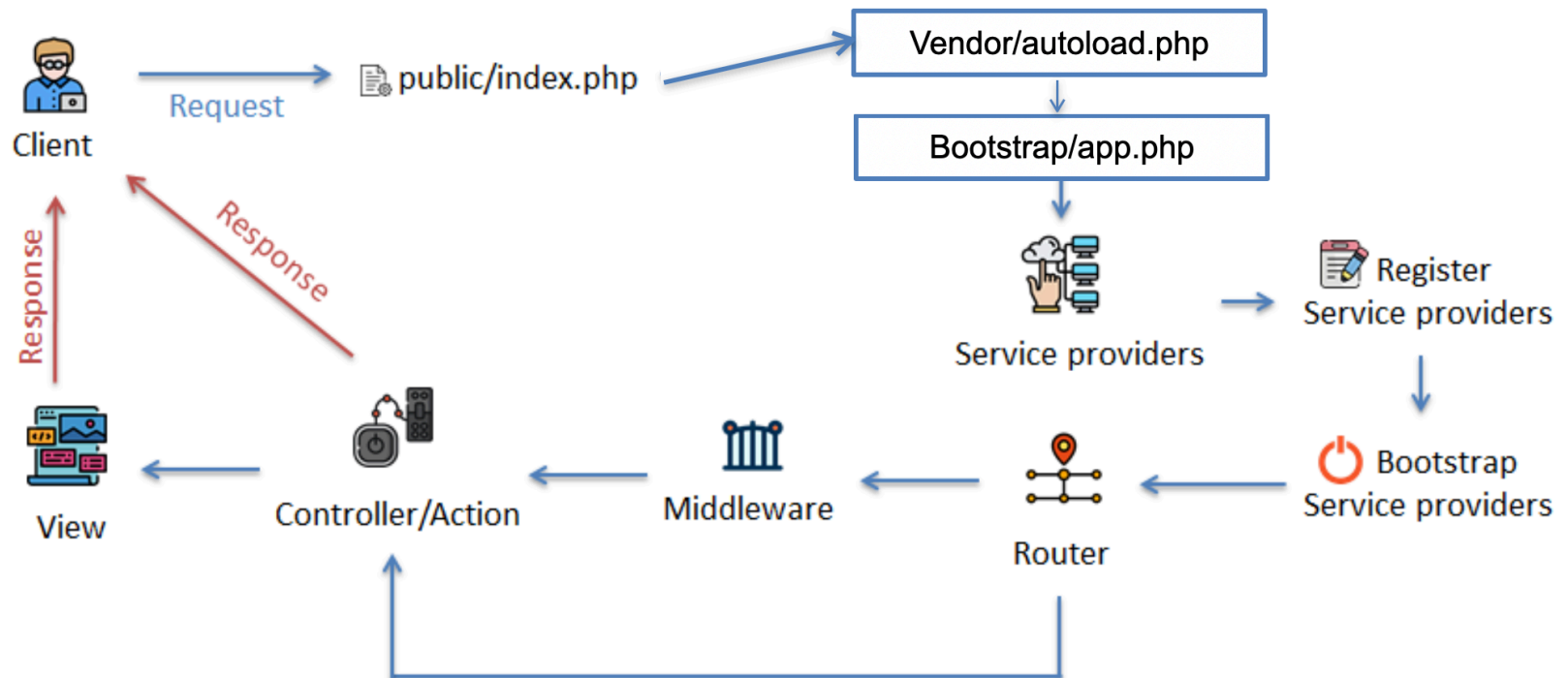
It enforces a separation between "business logic" from the input and presentation logic associated with a graphical user interface (GUI).





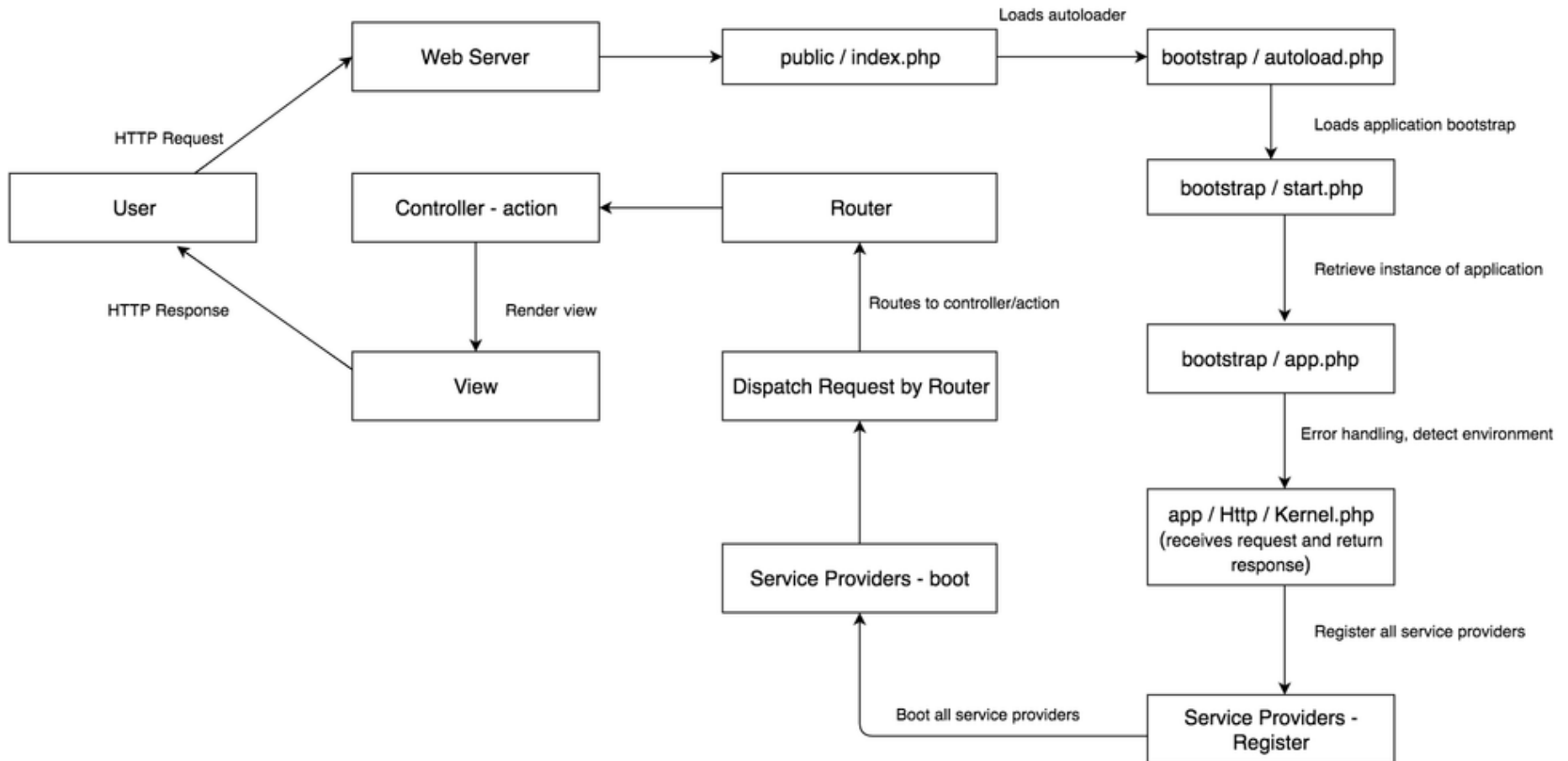
Lifecycle

Request Lifecycle Laravel





Request Life Cycle of Laravel





HOA SEN
UNIVERSITY

www.hoasen.edu.vn

LARAVEL BEST PRACTICES



Controller - Naming Conventions

- Controller name **MUST** start with a noun (in singular form) followed by the word “Controller”
 - Good

```
class ArticleController extends Controller {
```

- Bad

```
class ArticlesController extends Controller {
```

```
class wp_articlesController extends Controller {
```

```
class Articles extends Controller {
```



Model - Naming Conventions

- Model names **MUST** be in singular form with its first letter in uppercase
 - Good

```
class Flight extends Model {
```

- Bad

```
class Flights extends Model {
```

```
class flight extends Model {
```




Model - Naming Conventions

- hasOne or belongsTo relationship methods MUST be in singular form

- Good

```
class User extends Model{  
    public function phone() {  
        return $this->hasOne('App\Phone');  
    }  
}
```

- Bad

```
class User extends Model{  
    public function phones() {  
        return $this->hasOne('App\Phone');  
    }  
}
```

- Any other relationships other than above MUST be in plural form



Model - Naming Conventions

- Model properties should be in snake_case

- Good

```
$user->created_at
```

- Bad

```
$user->createdAt
```

- Methods should be in camelCase

- Good

```
class User extends Model{  
    public function scopePopular() {  
    }  
}
```

- Bad

```
class User extends Model{  
    public function scope_popular() { }  
}
```



Route - Naming Conventions

- Routes should be in plural form of the resource it is trying to manipulate and SHOULD be all lower-case

- Good

```
Route::get('/users', 'UserController@index');
```

```
Route::resource('photos', 'PhotoController');
```

- Bad

```
Route::get('/user', 'UserController@index');
```

```
Route::get('/UsersList', 'UserController@index'); }
```

```
Route::resource('PHOTO', 'PhotoController');
```

- Named Routes SHOULD use snake_case and dot notation

Good

```
Route::get('/user', 'UserController@active')->name('users.show_active');
```

Bad

```
Route::get('/user', 'UserController@active')->name('show-active-user');
```



Variable - Naming Conventions

- General rule for variable is it SHOULD be in camelCase
 - Good

```
$articlesWithAuthor
```
 - Bad

```
$articles_with_author
```
- Collection names SHOULD be descriptive and in plural form
 - Good

```
$activeUsers = User::active()->get()
```
 - Bad

```
$users = User::active()->get()  
$user = User::active()->get()  
$User = User::active()->get()
```
- Single Object SHOULD be descriptive and in singular form

Good

```
$activeUser = User::active()->first()
```

Bad

```
$users = User::active()->get()
```



View - Naming Conventions (1)

- You **SHOULD** use snake_case as file name of your Blade templates

- Good

```
show_filtered.blade.php
```

- Bad

```
showFiltered.blade.php  
show-filtered.blade.php
```

- You **MUST** not make non UI-related operations inside blade templates

- Good

```
// $api_results is passed by controller  
<ul>  
    @foreach($api_results as $result)  
        <li>{{ $result->name }}</li>  
    @endforeach  
</ul>
```



View - Naming Conventions (2)

- Bad

```
@php
    $api_results = json_decode(file_get_contents("https://
api.example.com"));
@endphp
<ul>
    @foreach($api_results as $result)
        <li>{{ $result->name }}</li>
    @endforeach
</ul>
```



Database Conventions (1)

- Table names **MUST** be in plural form and **MUST** be all lower-case

Good

```
class CreateFlightsTable extends Migration
{
    public function up()
    {
        Schema::create('flights', function (Blueprint $table) {
```

~~flight~~

- Pivot table names **MUST** be in singular model names in alphabetical order

Good

```
post_user
article_user
photo_post
```

~~posts_users
article_users
photos_post~~



Database Conventions (2)

- Table column names SHOULD be in snake_case without the model name

Good

```
username  
title  
thumb_url
```

Bad

```
UserName  
_title  
ThumbUrl  
post_title
```

- Primary Keys SHOULD be “id”
- Foreign keys MUST be singular model name with _id suffix

Good

```
User_id
```

Bad

```
userid  
siteid  
Memberid  
TransactionID
```


References

- <https://www.javatpoint.com/laravel>